

Slash commands

Slash commands are useful in several contexts:

General Benefits

1. **User-Friendly Interface** — Easier than typing full commands or navigating complex menus
2. **Command Discovery** — Often auto-complete suggestions show available commands
3. **Standardized Format** — Consistent /command syntax across different platforms
4. **Context-Aware Actions** — Quickly trigger specific operations without deep navigation

1. /explain — The Teacher

This command breaks down complex or confusing code and tells you exactly what it does step-by-step.

- **Real-Time Use Case:** You just joined a new company or downloaded an open-source project. You open a file and see a massive, terrifying regular expression (regex) or a complex database query that looks like hieroglyphics

`/explain`

Or

`/explain` Explain this like I am new to Java. Also explain the execution flow.

2. /fix — The Doctor

This command looks at broken, buggy, or crashing code and suggests a solution to patch it up.

- **Real-Time Use Case:** You run your web app and the terminal screams a massive red error at you: `TypeError: Cannot read properties of undefined (reading 'map')`. Your app is broken and you aren't sure why

`/fix`

Or

`/fix` Handle null values without changing the public function signature.

3. /tests — The Safety Net

Writing unit tests is incredibly important, but it can be repetitive and boring. `/tests` writes them for you.

- **Real-Time Use Case:** You just finished writing a neat custom function that calculates shopping cart discounts, including tax and coupon codes. Before you send it to production, your boss says, *"Make sure you write unit tests for this!"*

`/tests` Use Vitest. Cover empty input, invalid email and duplicate users.

4. /doc — The Secretary

Code needs documentation so other humans can read it. /doc generates comments and explanations for your functions.

- **Real-Time Use Case:** You wrote a great piece of code, but you know that if you look at it three months from now, you'll completely forget how it works. You need to add standard documentation blocks (like JSDoc) before submitting your Pull Request.

/setupTests — “Prepare the testing system”

/compact — “Summarize this long conversation”

/init — “Learn the rules of this project”

/startDebugging — “Configure the debugger”

/plan — Plan before changing files

/diff — Review what Copilot changed

/research — Investigate before implementing